## *Memory Efficient Doubly Linked List – Insert At Beginning –Space Complexity and Algorithm*

### *Program* :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Logic:
    // New Node's npx = NULL ^ current_head
    newNode->npx = XOR(nullptr, head);

    // If the list was not empty, we must update the old
head's npx
    if (head != nullptr)
    {
        // Old head's npx was: NULL ^ Next
        // We need it to be: NewNode ^ Next

        // 1. Decode the 'next' node of the current head
        Node *next = XOR(nullptr, head->npx);

        // 2. Update head's npx
        head->npx = XOR(newNode, next);
    }

    head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```

```
…..
        case 1:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtBeginning(data);
            break;
```

# *Space Complexity Analysis*:

***XOR Linked List (Insert At Beginning)***

1. ***Input Space Complexity***

   - ***Parameter – only Definition***:

     - ***Function Parameter***: ***int data***.

     - ***Analysis***: ***The data parameter is a single integer. It occupies a fixed amount of memory (typically 4 bytes)regardless of the size of the list.***

     - ***Complexity***: $O(1)$

2. ***Full Data Definition***:

   - ***Function Data***: ***The int data parameter + the existing XOR list.***

   - ***Analysis***: ***To perform the insertion, the function interacts with the existing list (specifically the current head).***
     ***If the list contains n nodes, the total*** input data ***context is proportional to n.***

- $\boldsymbol{Complexity}: \boldsymbol{O(n)}$.

# Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Auxiliary}} + \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Input}} = \boldsymbol{O(1)}.$$

- **Holistic Interpretation (Full-data input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Auxiliary}} + \underbrace{\boldsymbol{O(n)}}_{\boldsymbol{Input}} = \boldsymbol{O(n)}.$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

---

## *Algorithm*: *Insert at Beginning* (*XOR Doubly Linked List*)

***XORINSBEG***(***START***, ***ITEM***):

(((***Node*** ∗)***malloc***(***sizeof***(***Node***)) ***represents the availability of***

***space for the data***; ***let's name it AVAIL.***))

**1.** [***ALLOCATE NEW NODE***] ***Set*** ***NEWPTR*** := ***AVAIL***

**2.** [***MEMORY ALLOCATED***?]

***If*** ***NEWPTR*** = ***NULL***, ***then***:

***Write***: "***Error***: ***Memory allocation failed.***"

***Return and EXIT.***

[***End of If structure***]

**3.** [***COPY DATA***] ***Set*** ***INFO***[***NEWPTR***] := ***ITEM***

**4.** [***INITIALIZE NEW NODE'S POINTER***]

***Set*** ***NEXTXORPTR***[***NEWPTR***] := ***NULL*** ⊕ ***START*** ( ***This sets the new node's pointer field to the address of the current head, as its previous is NULL*** )

**5.** [***UPDATE OLD HEAD IF LIST NOT EMPTY***]

***If*** ***START*** ≠ ***NULL***, ***then***:

***a. Set*** ***NEXTTEMPPTR*** := ***NULL*** ⊕ ***NEXTXORPTR***[***START***] ( ***Decode the address of the node following the current head*** )

***b.*** ***Set*** $NEXTXORPTR[START] := NEWPTR \oplus NEXTTEMPPTR$ ***(Update the old head to point back to the new node and forward to its original successor)***

[***End of If structure***]

**6.** [***UPDATE HEAD POINTER***]

***Set*** $START := NEWPTR$ (***Update the global head to the newly created node***)

**7.** [***PRINT MESSAGE***]

***Write***: $ITEM$, " ***inserted at the beginning.***"

**8.** ***EXIT***

----------**********